BSc (Hons) in Computer Science & Artificial Intelligence

Course Work: Critical Review and Technical Baseline of a Machine Learning

Research Paper

Reviewed Paper:

"FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning"

Zhang, J., Liu, Y., Hua, Y., & Cao, J. (2024).

Publication: Proceedings of the AAAI Conference on Artificial Intelligence, 38(15), 16768-16776.

DOI: https://doi.org/10.1609/aaai.v38i15.29617

- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030
- Module : Machine Learning Related Application
- Supervisor / Module Leader : Mr.Ashan Arthnayaka
- Date of Submission: Oct 12, 2026

Table of Contents

A. Problem Formulation..... 3
A.1 Introduction to the Problem..... 3
A.2 The Core Machine Learning Problem in Formal Terms..... 8
A.3 Why Conventional Federated Averaging Fails Under Model Heterogeneity..... 10
A.4 Statistical Heterogeneity..... 11
A.5 Model Heterogeneity..... 13
A.6 The client training objective and prediction rule..... 15
A.7 The Four Datasets: An Overview..... 16
A.8 What the Dataset Selection Allows the Paper to Investigate..... 23
A.9 Input Representation and Learning Type..... 24
A.10 Prototype Statistics and Their Limitations..... 25
A.11 Normality Assumptions..... 26
A.12 Defining Successful Performance..... 26
A.13 Scope, Privacy, and Boundaries of the Problem..... 28
B. Methodological Justification..... 29
B.1 Introduction to the Methodology..... 29
B.2 Literature Review and Justification..... 30
B.3 Why the Authors Selected Trainable Global Prototypes..... 35
B.4 Why the Authors Selected ACL..... 37
B.5 Local Training Objective and Prediction..... 39
B.6 Client Models Used in FedTGP..... 39
B.7 FedTGP Methodology Diagram..... 43
C. Data Pipeline Analysis..... 44
C.1 Introduction..... 44
C.2 Overview of the Datasets..... 45
C.3 Preprocessing & Feature Scaling..... 45
C.4 Label Encoding..... 47
C.5 Class Imbalance and Client Data Distribution..... 48
C.6 Train-Test Splitting..... 49
C.7 Dataset Choice for Phase 2..... 50
References..... 50

Phase 1: Conceptual Mapping and Methodological Deconstruction

A. Problem Formulation

A.1 Introduction to the Problem

Why Federated Learning Instead of Centralized Learning

Traditional centralized machine learning models usually get better when they can learn from more data. A model that has seen thousands of cat images will recognize a new cat far more reliably than one trained in only a handful of examples. This simple fact has driven the success of modern deep learning, where massive datasets are gathered into centralized repositories and used to train increasingly powerful networks. Particularly, a common example is ImageNet, which has a vast set of labelled images that have been used to train models that are common in computer vision .

The centralized approach is suitable when the following three conditions are satisfied; First, the data can be legally and ethically moved to one location. Second, the data is likely to be shared by the individuals or group of individuals that have the data. Third, the communication and storage infrastructure can handle the required communication and storage. When all three are true, centralized training is simple, efficient, and well understood.

However, in reality, these circumstances do not always work out. When this does occur it is impossible or it is very problematic to have centralized learning. The transfer of personal data between organizations and between countries is to some extent limited by legal and regulatory issues and in certain situations even barred. Competitive and economic factors can also be of concern , such that even if permitted by law, organizations may not be willing to provide transfer of their data where it has strategic or commercial value. Collecting data at the network edge is costly, slow, or even difficult when it is generated there due to practical and infrastructure

limitations. The ethical dimension is enhanced by the considerations of privacy: if data transfer is allowed, people might not want their personal data to be centralized.

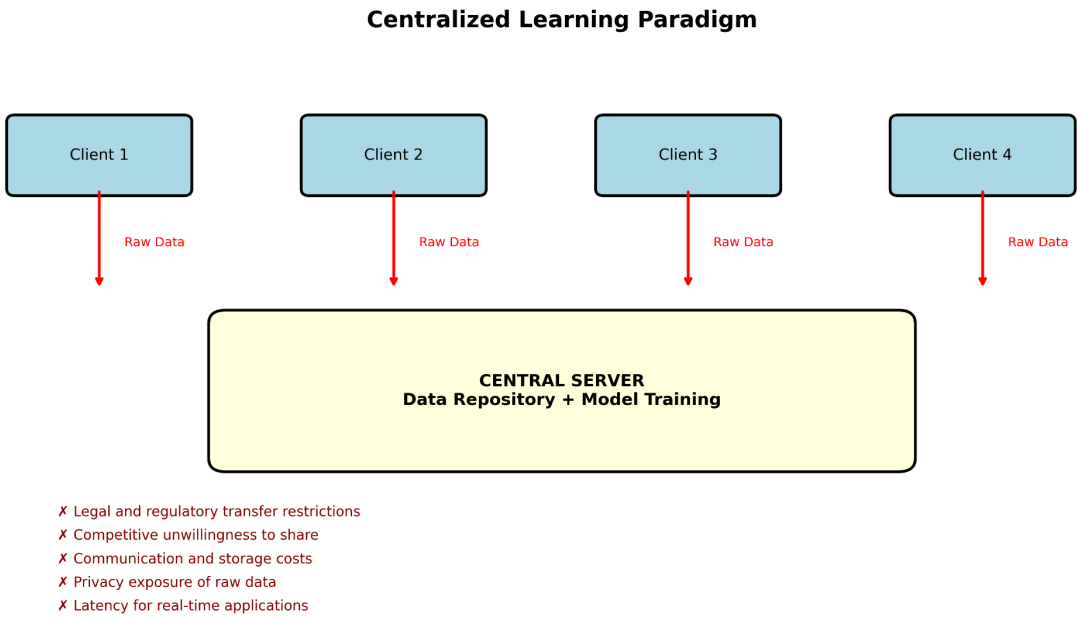


Figure A.1. The centralized learning paradigm. Raw data from all clients is transferred to a central server, where a single model is trained. This approach faces legal, competitive, practical, and privacy barriers.

Federated Learning offers a different approach. Instead of moving data to the model, it moves the model to the data. A local model is trained on its own private data by each participant (client). The client then shares only selected learning information with a central server. The server combines these contributions in some way, and the improved global knowledge is sent back to the clients for the next round. The process continues for a specified number of communication rounds or until a stopping criterion is reached [2].

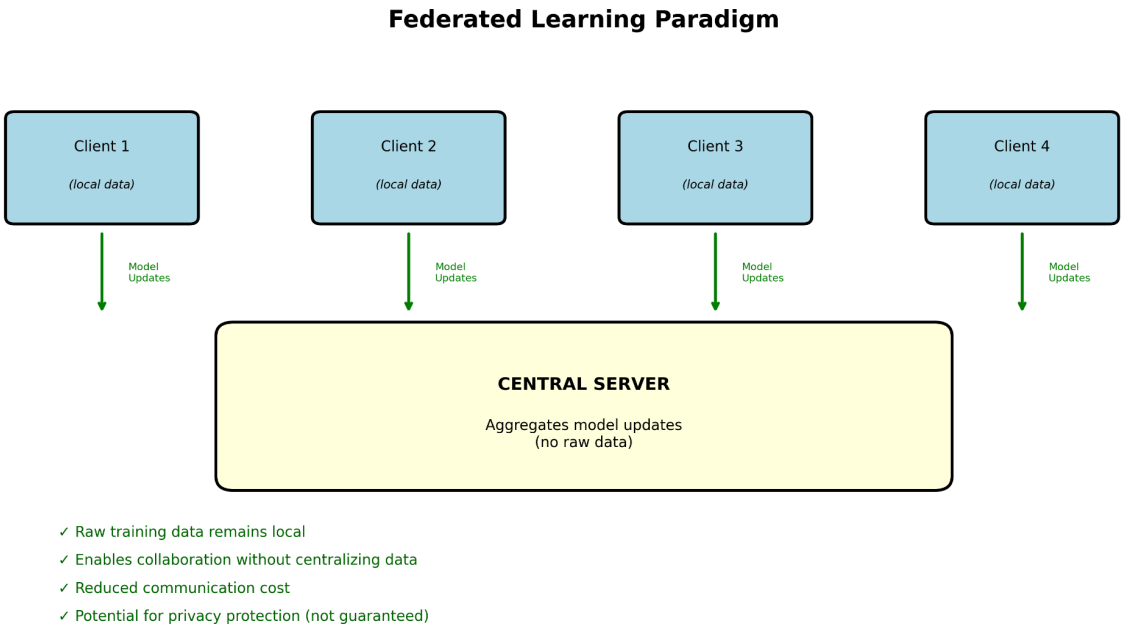


Figure A.2. The federated learning paradigm. Clients train local models on their private data and share only model updates with the central server. Raw training data remains local, though communicated information may still present privacy risks [2].

One of the most popular federated learning algorithms is known as Federated Averaging or FedAvg. In FedAvg, each client trains a local copy of the same neural network architecture on its own data. After local training, each client sends its updated model parameters to the server. The server then computes a weighted average of these parameters, where the weight for each client is proportional to the number of training examples it holds. This weighted average becomes the new global model, which is broadcast back to all clients for the next round [2].

FedAvg can handle non-IID client data, and the original evaluation provides for unbalanced and non-IID cases. Strongly different local distributions can, however, render optimization and generalization more challenging. The major restriction on its direct use in architecture is that there must be compatible model parameters to average [2].

Conventional FedAvg cannot directly average complete models with incompatible parameter structures. Clients may require architectures because their resources or application needs differ.

FedTGP investigates collaboration under this architectural variation together with differences in local data distributions [1], [2].

The paper FedTGP:Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning by Zhang, Liu, Hua and Cao directly addresses both these limitations. The authors explore the situation where clients have different data distributions and also use different neural network architectures. Their proposed solutions involve class prototypes–compact numerical representations of categories in a learned feature space. Instead of sharing complete model parameters or raw images, clients compute the average feature vector for each class in their local data and send only these prototypes to the server. The server then learns improved global prototypes that guide the local training of all clients [1] .

The Problem, Solution and the Research Gap

FedTGP was not the first method to address heterogeneity in federated learning, but earlier approaches left a specific gap that the authors set out to fill [1]. To understand this gap, it is necessary to clearly outline the problem, what could and could not be achieved with available methods, and what FedTGP offers as a solution. Technical details of individual methods are located in Part B where the methodology is deconstructed. This section provides only the conceptual framing.

The problem : Federated Learning enables clients to learn together without sharing their actual data. When clients have different data distributions (statistical heterogeneity) and incompatible neural network architectures (model heterogeneity), the standard parameter-averaging methods cannot be applied, since the parameters of different architectures are not compatible. Prototype-based methods such as FedProto solve the communication problem by sharing class prototypes–fixed-dimensional vectors that are independent of the network that produced them. But FedProto aggregates these prototypes by weighted averaging, which the fedTGP authors identify as having three specific limitations [1]:

1. Privacy leakage: The weighted average protocol requires clients to upload class distribution information as weights, thereby revealing sensitive information about clients' private data.

2. Prototype margin shrink: Prototypes from heterogeneous clients have diverse scales and separation margins. Averaging produces global prototypes with smaller margins than the well-separated prototypes, reducing their discriminability [1].

3. Model quality is ignored: In averaging, the weights are determined by the quantity of the data. A prototype from a poorly trained model may receive a large weight if that client has more data, which can impair clients that previously produced well-separated prototypes.

The solution: FedTGP replaces simple averaging with Trainable Global Prototypes (TGP) learned on the server using Adaptive-Margin-Enhanced Contrastive Learning (ACL) [1]. The server maintains a trainable vector for each class and passes it through a small neural network F (two fully connected layers with ReLU) to produce the global prototype. The contrastive objective encourages global prototypes to align with local prototypes of the same class and remain separated from competing classes [1]. The adaptive margin changes according to the current geometry of the client prototypes.

The research gap: Prototype-based learning helps communication across heterogeneous models, but direct averaging does not explicitly optimize the discrimination of global prototypes. FedTGP addresses this gap by learning global prototypes with an alignment and separation objective [1].

The figure below summarizes the conceptual relationship.

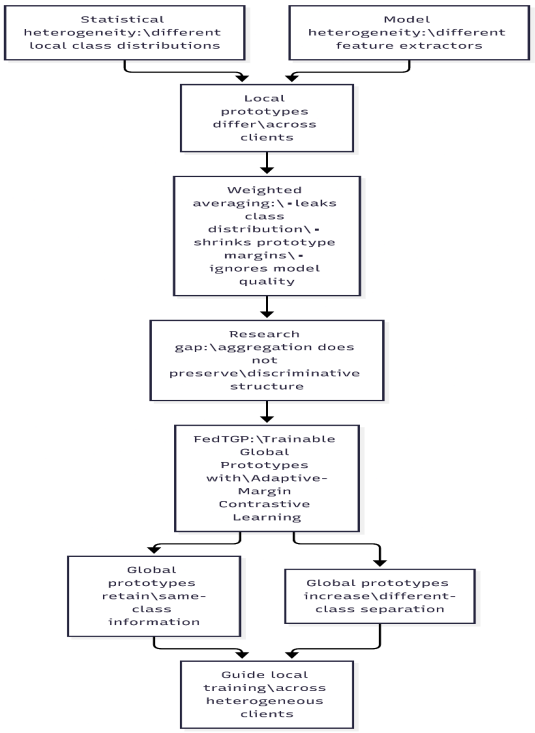


Figure A.3 The research gap and FedTGP's response. Heterogeneity causes local prototypes to differ; weighted averaging has three specific limitations; FedTGP learns global prototypes with an adaptive-margin contrastive objective that preserves discriminative structure [1].

A.2 The Core Machine Learning Problem in Formal Terms

The task FedTGP addresses is supervised, single-label, multiclass image classification under statistical and model heterogeneity [1]. Each client i holds a private dataset.

$$D_i = \{(x_{i,j}, y_{i,j})\}_{j=1}^{n_i}$$

Where $X_{i,j}$ is an input image and $y_{i,j} \in \{0, 1, \ldots, C-1\}$ is its correct class label. The number of clients is M, and the total number of classes is C. The goal is for each client to learn a local classifier $h_i(f_i(X))$ that predicts the correct label for unseen images drawn from the same distribution as its local data.

This is a supervised learning problem because every training image comes with a label. It is a single-label problem because each image belongs to exactly one category. It is a multiclass

problem because $C > 2$. It is a federated learning problem because the training data is distributed across clients and is never centralized.

The problem becomes substantially harder because of two forms of heterogeneity.

Statistical Heterogeneity means that different clients have different data distributions. Formally, if $P_i(X, Y)$ denotes the joint distribution of images and labels on client i, then for two different clients i and j,

$$P_i(X, Y) \neq P_j(X, Y)$$

This joint distribution can be decomposed using the chain rule of probability as

$$P_i(X, Y) = P_i(Y).P_i(X|Y)$$

The term $P_i(Y)$ describes how common each category is on client i, while $P_i(X \mid Y)$ describes what images of each category look like. Different clients may differ in either or both.

Model heterogeneity means that clients use different neural network architectures. The paper's main experiments use eight different feature extractors: a four-layer CNN, GoogLeNet, MobileNetV2, ResNet18, ResNet34, ResNet50, ResNet101, and ResNet152 [1]. These architectures vary widely in depth and computational cost [1]. A model f_i on client i and f_j on client j may have entirely different parameter spaces, which makes direct parameter averaging impossible.

The combination of these two forms of heterogeneity creates a fundamental tension. Statistical heterogeneity means that a single global model may not serve all clients well, because each client's optimal decision boundary depends on its own data distribution. Model heterogeneity means that even if a single global model were desirable, it could not be built by averaging the clients' parameters. The challenge is therefore to find a form of shared knowledge that is both robust to distributional differences and independent of architectural details.

A.3 Why Conventional Federated Averaging Fails Under Model Heterogeneity

The expression below shows the full-participation case; with partial participation, aggregation is restricted to the selected clients. In FedAvg, the server maintains a global model with parameters θ [2] . At each communication round t, a subset of clients is selected to participate. Each selected client i receives the current global parameters $\theta^{(t)}$, performs several epochs of local training on its private data, and produces updated parameters $\theta_i^{(t+1)}$. The server then aggregates these updates using the weighted average

$$\theta^{(t+1)} = \sum_{i=1}^{M} \frac{n_i}{N} \theta_i^{(t+1)}, \quad N = \sum_{i=1}^{M} n_i \qquad \text{(A.4)}$$

Where n_i is the number of training examples on client i.

The weighting coefficient n_i/N gives greater influence to clients with more data. This makes sense when all clients are sampling from the same underlying distribution, because a client with more data provides a more reliable estimate of the shared true gradient. However, the operation assumes that the parameters $\theta_i^{(t+1)}$ different clients are compatible. Compatibility requires not only that the parameter vectors have the same dimension, but also that corresponding coordinates play corresponding roles in the model's computation.

A four-layer CNN and a ResNet152 violate the requirement in two ways. First, their parameter [1] vectors have different dimensions: the CNN has far fewer parameters than the ResNet [1]. Therefore, mathematically the averaging operation in Equation(A.4) is undefined. Secondly, if the dimensions were to match, the parameters would not be systematically aligned. The first layer of a four-layer CNN and the first layer of a ResNet152 can be convolutional, but they are different layers at different scales, with different receptive fields, and with different following processing [1]. The average of them will be a parameter vector that is not related to any meaningful feature extractor.

This incompatibility makes it impossible to directly use the conventional FedAvg approach to average the various client models used in FedTGP. Thus, communication between the various components of the collaboration needs to be independent of the parameter structure of the clients [1], [2].

One method to achieve this is through sharing prototypes. A prototype is a vector that is defined in a common feature space; the dimension of the prototype can be chosen independently of the architecture which generates it. With all clients generating a 512-dimensional feature vector prior to their final classification layer regardless of whether they are using a small CNN or a deep ResNet, their class prototypes will be 512-dimensional vectors. By default, FedTGP is set to use the value of $K = 512$. The dimensional compatibility allows prototype communication between different architectures [1].

Dimensional compatibility does not necessarily imply semantic alignment, though. For instance, the same coordinate can be used to highlight colour in one client, and texture in another. Distances between these vectors can be computed, but it is not enough that they are equal in dimension to justify meaningful comparisons between classes. This is a deeper challenge addressed by FedTGP: learning global prototypes that remain useful across clients whose local feature spaces are organized differently. Global prototypes provide shared guidance to help align and separate classes during local training [1].

A.4 Statistical Heterogeneity

This describes any situation where the data distributions of different clients differ. The paper focuses mainly on label-distribution heterogeneity, which means that different clients see different proportions of the available classes.

In the most extreme case, called the pathological non-IID setting in the paper, each client receives examples from only a small subset of the available classes [1]. For CIFAR-10, which has 10 classes, each client receives examples from only two classes. This means that a client assigned the classes “cat” and “dog” has no direct local supervised evidence for “airplane”, “automobile”, “bird”, “deer”, “frog”, “horse”, “ship”, or “truck”. Its local model can only learn

to distinguish between cats and dogs from its own data, and it has no direct local evidence about what the other eight classes look like. Shared representation may support transfer, but successful recognition is not guaranteed.

The paper also studied a less extreme but less restrictive condition called the practical non-IID setting. Here, the authors use a Dirichlet distribution to allocate class examples to clients. The Dirichlet distribution is a probability distribution over the simplex–that is over vectors of non-negative numbers that sum to one. For each class c, an allocation vector [1]

$$q_c = (q_{c,1,\ldots,} q_{c,M}) \sim Dirichlet(\beta,\ldots,\beta)$$

is drawn, where $q_{c,i}$represents the fraction of class c's examples assigned to client i.

The constraint

$$q_{c,i} \geq 0, \sum_{i=1}^{M} q_{c,i} = 1$$

ensures that the fractions are valid proportions.The concentration parameter β controls how uneven the allocation is. When β is large, the Dirichlet distribution concentrates near the uniform vector $(1/M,\ldots,1/M)$, producing relatively balanced allocations. When β is small, the distribution concentrates near the corners of the simplex, producing allocations where a few clients receive most of the examples of a class while many clients receive almost none. The paper uses β = 0.1, which produces highly uneven allocations [1].

For a symmetric Dirichlet distribution with parameter β and M categories, the expected value of each component is

$$E[q_{c,i}] = \frac{1}{M},$$

And the variance is

$$Var(q_{c,i}) = \frac{M-1}{M^2(M\beta+1)}$$

With M = 20 clients and β = 0.1, these become

$$E[q_{c,i}] = 0.05,\ Var(q_{c,i}) = \frac{19}{400\times3} = \frac{19}{1200} \approx 0.01583,$$

And the standard deviation is

$$SD(q_{c,i}) \approx 0.1258$$

The standard deviation is more than twice the mean, which indicates that the allocation is highly uneven. In a typical draw, some clients will receive a large fraction of a class's examples while others receive very few. This can create substantial local imbalance even when the original dataset is globally balanced.

It is important to note that the expected value being $1/M$ does not mean that every realized allocation will be balanced. The Dirichlet distribution is a probability distribution, and a single draw can produce a highly skewed allocation. The analysis of the method should therefore consider the realized client by-class count matrix $N = (n_{i,c})$, where $n_{i,c}$ is the number of examples of class c on client i. This matrix determines the actual difficulty of the learning problem.

Each client's local empirical risk is

$$\widehat{R}_i = \frac{1}{n_i} \sum_{j=1}^{n_i} l(h_i(f_i(X_{i,j})), y_{i,j})$$

This estimates the expected risk $R_{i,}$ under client i's data distribution P_i. Because distributions differ,minimising the average of these empirical risks does not necessarily minimise every client's expected risk. This challenge motivates personalized and prototype-based approaches.

A.5 Model Heterogeneity

Model Heterogeneity arises when clients use different neural network architectures, either because they have different computational resources or because they have chosen different designs for their own reasons.

The paper's main experimental group, called HtFE8, includes eight different feature extractors. Those architectures span a wide range of depths and computational costs.

From a federated learning perspective, the critical fact about these architectures is that their parameter spaces are not compatible. A parameter vector θ_i for client i and θ_j for client j cannot be combined through simple arithmetic operations unless they have the same dimension and corresponding coordinates have corresponding meanings.

The paper's solution is to separate each client's model into two parts: a feature extractor f_i and a classification head h_i. The feature extractor maps an input image X to a learned embedding

$$Z_i = f_i(X; \theta_i) \in R^K,$$

And the classification head maps this embedding to class scores

$$S_i = h_i(Z_i; w_i) \in R^C.$$

The paper reports using an average-pooling layer after the feature extractors and sets the common feature dimension to $K = 512$ [1]. The exact architecture modifications should be established from the implementation rather than inferred from this description.

This architectural choice is what makes prototype communication possible. A prototype P_i^c for class c on client i is defined as the average embedding of all local examples of that class:

$$P_i^c = \frac{1}{n_{i,c}} \sum_{(x,y) \in D_{i,c}} f_i(X; \theta_i), n_{i,c} > 0,$$

Where $D_{i,c} = \{(x, y) \in D_i : y = c\}$ and $n_{i,c} = |D_{i,c}|$. Since every prototype is a 512-dimensional vector, prototypes from different clients can be compared, averaged, or otherwise combined even if the clients' full models are completely different.

However it is not enough that they are compatible in dimension. It is possible that the fifth coordinate of A's embedding is not the same as the fifth coordinate of B's embedding. The architectures learn different internal representations and the representations are not necessarily aligned in the feature space. That's why FedTGP doesn't just take the local prototypes and average them to get the global prototypes [1]. Instead, it learns global prototypes on the server side with a contrastive objective that explicitly pushes prototypes of the same class toward being

close and prototypes of the different classes to be far apart [1]. The learning process is designed to find a common representation space in which the prototype of various clients become comparable.

Class prototypes enable client/server systems to share class data regardless of the client/server architectures. Averaging heterogeneous prototypes, however, may cause the classes to be less separated. However, FedTGP overcomes this limitation by learning global prototypes using an explicit alignment and separation objective [1].

A.6 The client training objective and prediction rule

The global prototypes learned by the server are used to guide the local training of each client [1]. The paper formulates local guidance using class prototypes:

$$L_i = E_{(x,y)} \sim D_i l(h_i(f_i(X;\theta_i);w_i),y) + \lambda E_{c\sim c_i}\phi(P_i^c,\widehat{P}^c).$$

The first term is the standard classification loss, which encourages the local model to predict the correct label for each image. The second term is the prototype alignment loss, which encourages the local prototypes to be close to the corresponding global prototypes. The coefficient λ controls the strength of the prototype guidance [1]. The paper uses $\lambda = 0.1$ by default. The prototype term has a coefficient of 0.1, but its actual influence depends on the scales, reductions and gradients of both loss terms [1]. The coefficient alone does not show which term dominates training.

At inference time, FedTGP uses a nearest-prototype rule rather than the classification head. For a test image X, the predicted class is [1]

$$\widehat{y}(X) = argmin_c \left\| f_i(X;\theta_i) - \widehat{P}^c \right\|_2$$

The feature extractor produces an embedding, and the closest global prototype determines the prediction. The classification head is used during training to provide supervised feedback, but it is not used at test time.

A.7 The Four Datasets: An Overview

FedTGP is evaluated on four image classification datasets: CIFAR10, CIFAR-100, Flowers102, and Tiny-ImageNet [1]. These datasets were chosen to provide a range of difficulty levels, class counts, and image resolutions, allowing the authors to assess whether their method generalized beyond a single benchmark.

The following table summarizes the key characteristics of each dataset. These are the properties of the original datasets as published by their creators, not the client-level partitions used in the paper.

Dataset	Modality	Classes	Original Resolution	Training Images	TestImages
CIFAR-10	RGB Images	10	32×32	50,000	10,000
CIFAR-100	RGB Images	100	32×32	50,000	10,000
Flowers102	RGB Images	102	Variable	1,020	6,149
Tiny-ImageNet	RGB Images	200	64×64	100,000	10,000

Table A.1. overview of the datasets used to evaluate FedTGP.

The CIFAR dataset specifications are documented by Krizhevsky, Nair, and Hinton [3]. The Flowers102 dataset is described by Nilsback and Zisserman [6], with category statistics available from the Visual Geometry Group [15] and standard split sizes documented in TensorFlow Datasets [16]. Tiny-ImageNet is described in a Stanford CS231n project report [17].

It is important to distinguish between the official dataset splits and the client-level splits used in FedTGP. The paper reports that each client's private data is divided into 75% for training and

25% for testing [1]. This means that the effective training and test sets in the federated experiments are different from the official dataset splits. All comparisons between FedTGP and other methods should use the same client-level splits to ensure fairness [1].

CIFAR-10 Dataset

CIFAR-10 contains 10 mutually exclusive categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each image is a 32×32 pixel RGB image, so the raw input dimension is

$$D = 32 \times 32 \times 3 = 3,072$$

The target variable is a categorical label

$$Y \in \{0, 1, \ldots, 9\}$$

These labels identify categories rather than numerical quantities. Applying mean squared error directly to integer class IDs would introduce an artificial ordering. Cross-entropy is appropriate for categorical predictions, although squared error applied to one-hot targets does not introduce that same ordering.

The official training set contains 50,000 images, with exactly 5,000 images per class. The empirical class prior is therefore [3]

$$\widehat{P}^{c} = \frac{5,000}{50,000} = 0.1$$

For every class c. The mean class count is 5,000, and the variance across the ten class counts is exactly zero, because every class has the same number of training examples.

This global balance does not mean that the data is balanced at the client level. After federated partitioning, a client may have many examples of one class and few or none of another. The global balance is a property of the entire dataset, while the local balance is a property of each client's partition. The two are not the same, and the paper's non-IID partitioning deliberately creates local imbalance while preserving global balance.

The class-label entropy of the training distribution is

$$H(Y) = -\sum_{c=1}^{C} P_c \ln P_c$$

For the balanced ten-class distribution with $P_c = 0.1$ for all c,

$$H(Y) = -10 \times 0.1 \times \ln(0.1) = \ln(10) \approx 2.303\ nats.$$

Label entropy measures uncertainty in the label distribution. CIFAR-10's balanced training distribution reaches the maximum value of $\ln(10)$. This is a calculation from its documented class counts, rather than a result reported by FedTGP [3].

Uniform random guessing has an expected accuracy of 10%. This provides a chance reference, but it is not an adequate trained baseline.

CIFAR-10 provides a manageable 10-category benchmark with small color images [3]. Some categories such as cats and dogs or deer and horses, can be visually similar and difficult to distinguish. The low image resolution also limits the fine detail available to the model.

CIFAR-100 Dataset

CIFAR-100 uses the same image format as CIFAR-10: 32×32 pixel RGB images. However, it contains 100 fine-grained categories rather than 10 coarse categories. Each Category has 500 training images and 100 test images in the official split [3].

The target variable is

$$Y \in \{0, 1, \ldots, 99\}$$

The raw input dimension remains

$$D = 3{,}072$$

Thus, the input size is unchanged while the label space increases tenfold. This makes CIFAR-100 substantially more difficult than CIFAR-10, because the model must distinguish between 100 categories using the same number of input features [3].

For the official training distribution, each class has 500 images, so the empirical class prior is [3]

$$P_c = \frac{500}{50{,}000} = 0.01$$

for every class. The class-label entropy is

$$H(Y) = ln(100) \approx 4.605\ nats.$$

The expected accuracy of uniform random guessing is 1/100 = 1%

CIFAR-100 also provides 20 superclass labels, which group the 100 fine-grained classes into broader categories [3]. However, the task considered in FedTGP uses the 100 fine-class labels, not the superclass labels [1].

CIFAR-100 has fewer official training images per category than CIFAR-10. An illustrative equal division among 20 clients would provide 25 images per category per client for CIFAR-100, compared with 250 for CIFAR-10. These are not FedTGP's actual non-IID counts, but they show how limited local class support can affect prototype estimation.

Flowers102 Dataset

Flowers102 is a fine-grained image classification dataset containing 8,189 images across 102 [15] flower categories. The number of images per category varies between 40 and 258, so the dataset is naturally imbalanced [15].

The source images have variable spatial dimensions, so preprocessing must make them compatible with the chosen batching procedure.

The target variable is

$$Y \in \{0, 1, \ldots, 101\}$$

The classification problem is fine-grained, meaning that the differences between categories are subtle. Different flower species may have similar shapes, colors, and textures, while examples of the same species can vary in pose, lighting, scale, and background [15]. This creates two related

challenges: high within-class variance (examples of the same class look different) and high between-class similarity (examples of different classes look similar). A useful feature representation must preserve the distinctions between species while tolerating variations that do not change the label.

The following statistics are calculated from the complete original category-count table. They describe the dataset as published, not the client level partitions used in FedTGP [15].

The average category count is

$$\overline{n} = \frac{8{,}189}{102} \approx 80.28$$

The population variance across all category counts is

$$\sigma_n^2 = \frac{1}{102}\sum_{c=1}^{102}(n_c - \overline{n})^2 \approx 1{,}941.22$$

The corresponding standard deviation is

$$\sigma_n \approx 44.06$$

The coefficient of variation is

$$CV = \frac{\sigma_n}{\overline{n}} \approx 0.549$$

This means that the standard deviation of category counts is approximately 54.9% of their mean. In other words, the category sizes vary substantially relative to their average.

The largest-to-smallest count ratio is

$$IR = \frac{258}{40} = 6.45$$

The largest category has 6.45 times as many images as the smallest, indicating unequal support for estimating class representations [15].

Using the empirical class proportions $p_c = n_c/8,189$ the label entropy is approximately [15]

$$H(Y) \approx 4.500\, nats,$$

Compared with the maximum possible entropy for 102 classes,

$$ln(102) \approx 4.625\, nats.$$

The entropy is close to the maximum, which means that the distribution is not extremely concentrated on a few classes [15]. The imbalance ratio emphasises the extreme counts, while entropy summarizes the entire distribution [15]. Both are useful, but they capture different aspects of the imbalance.

Uniform random guessing over 102 categories has an expected accuracy of approximately 0.98%. Always predicting the largest category over the complete dataset would give approximately 3.15% accuracy. These are illustrative calculations, not baseline results on FedTGP's test partitions.

The standard dataset split has 1,020 training images, 1,020 validation images, and 6,149 test images [16]. FedTGP's client-level 75/25 split is a different experimental arrangement. The two should not be confused [1].

Flowers102 is useful for studying prototype learning under fine-grained visual similarity and unequal category support [15]. The original dataset imbalance and the imbalance introduced by client partitioning are separate effects, and both should be considered when evaluating the method's performance.

Tiny-ImageNet Dataset

Tiny-ImageNet contains 200 categories with 64×64 pixel RGB images. The training set contains 100,000 images, with 500 images per category [17]. The target variable is

$$Y \in \{0, 1, \ldots, 199\}.$$

The raw input dimension is

$$D = 64 \times 64 \times 3 = 12,288.$$

This is four times the CIFAR input dimension:

$$\frac{12{,}188}{3{,}072} = 4$$

The larger resolution gives the model spatial data that can be used to make fine-grained distinctions. The time required to process larger images, though, varies depending on the architecture, convolution size, strides and pooling operations. The more parameters in the model, or floating-point operations per image, the longer it will take to train and evaluate the model.

During a 200-class setting, each prototype should be distinguishable from a larger set of competing categories. This is a more challenging test of the capability of the method to learn discriminative global prototypes. If the method is scalable to 200 classes, there is a higher probability that it can be applied in real life applications involving many categories [1].

FedTGP cites Chrabaszcz, Loshchilov, and Hutter when referring to Tiny-ImageNet. That work [1], [4], [17] describes a downsampled ImageNet variant retaining the original ImageNet class structure, rather than the same 200-class Tiny-ImageNet resource. This is a dataset-citation clarity issue. It does not independently demonstrate that the authors used incorrect data, but it does mean that readers should be careful when tracing the provenance of the dataset [17].

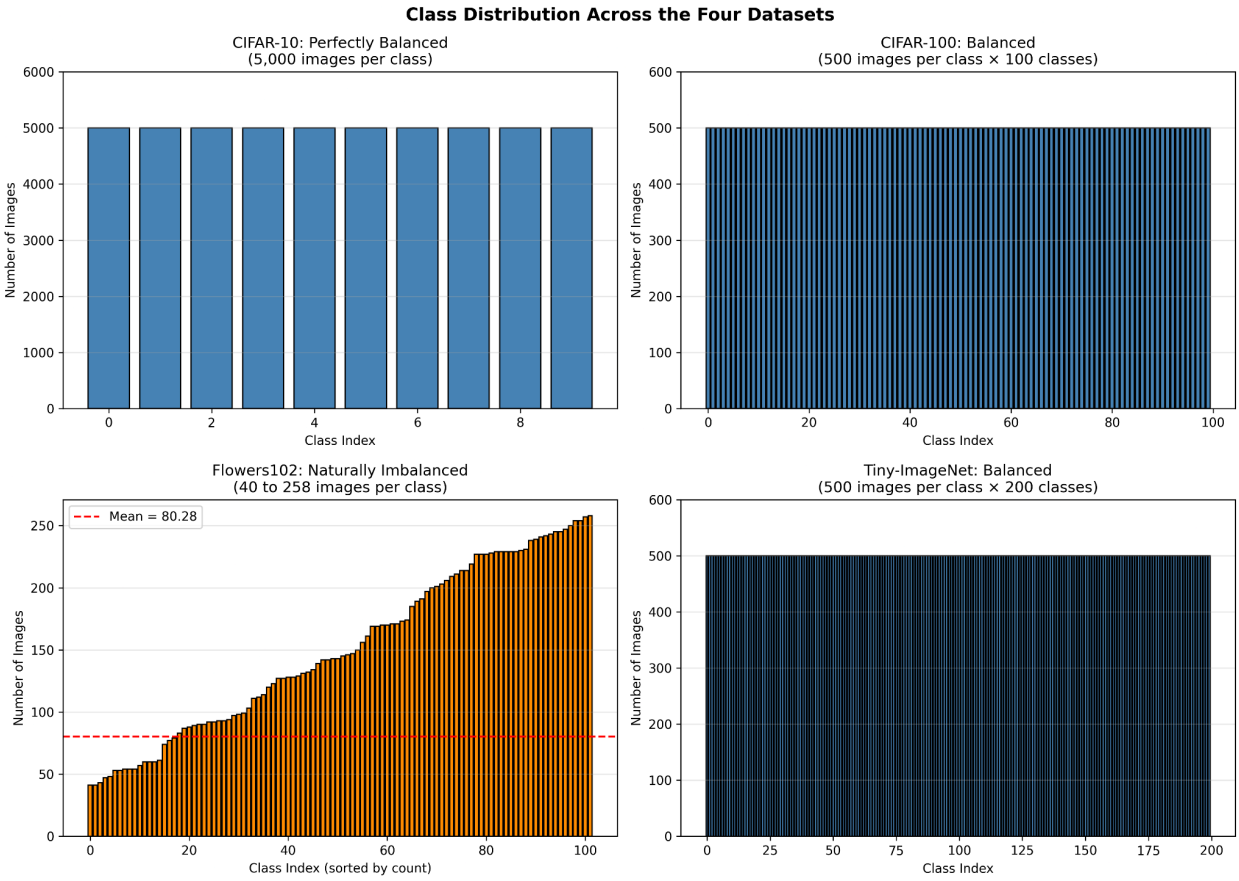


Figure A.4: Original category counts. CIFAR and Tiny-ImageNet use official training counts: Flowers102 uses complete dataset counts.

A.8 What the Dataset Selection Allows the Paper to Investigate

The four datasets allow for the evaluation of the method under various conditions,giving a wide range of assessment of the capabilities of the method.

CIFAR-10 provides a small, balanced label space. CIFAR-100 increases the category count while retaining the same image dimensions and total training-set size. Flowers102 introduces fine-grained visual similarity and unequal original category counts. Tiny-ImageNet combines more categories with larger images.

This selection will be used to review if there are benefits beyond the one benchmark. If FedTGP outperforms the other methods on all 4 datasets, this is better than just outperforming them on the dataset [1].

Evaluation across multiple datasets provides broader evidence than evaluation on a single benchmark.The four benchmarks are all related to image classification.These image-classification results do not establish performance on text, audio, volumetric medical images or every real-world federation.

This paper does not show that the four datasets are the most appropriate ones.Their methodological relevance is in the differences they offer in the domain assessed.

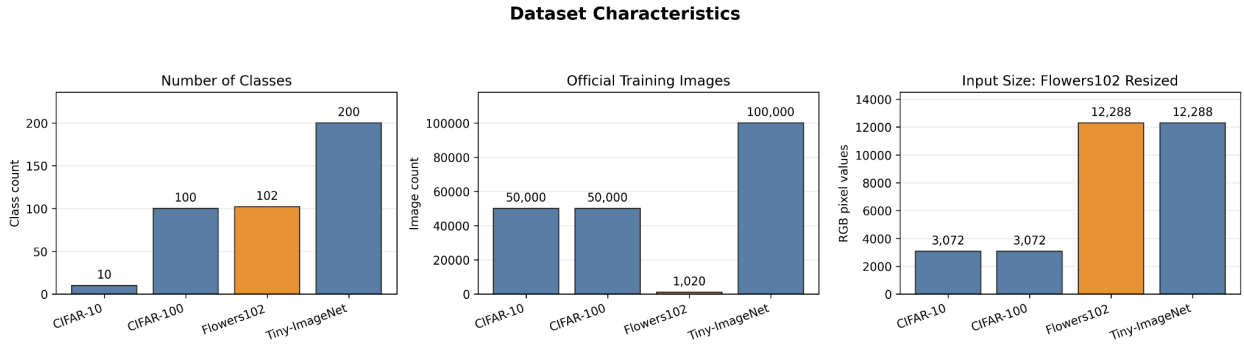


Figure A.5: Comparison of class counts, official training-set sizes and RGB input dimensions. Flowers102's input dimension assumes resizing to 64 × 64; its original image dimensions vary [3], [5], [16], [17].

A.9 Input Representation and Learning Type

The inputs to the model are 2-dimensional RGB images, represented in channel-first tensor format as

$$X \in R^{3\times H\times W}.$$

The three channels contain red, green, and blue information. They do not represent spatial depth. A 3-dimensional image would be a volumetric scan, such as a CT or MRI image, where the third

dimension represents depth within the object. CIFAR images have $H = W = 32$, giving 3,072 pixel values per image. Tiny-ImageNet images have $H = W = 64$, giving 12,288 pixel values.

Each client's feature extractor transforms these pixels into a learned representation. The paper sets the default feature dimension to $K = 512$. The feature extractor is specific to client i, and its parameters θ_i are not shared with the server or other clients.

The target is one categorical label per image:

$$Y \in \{0,\ldots, C - 1\}.$$

FedTGP therefore studies supervised, single-label multiclass classification. The labels guide both the classification objective and the formation of class prototypes that represent the same category and which represent competing categories [1]. Federated learning describes the distributed training arrangement, not a separate supervised or unsupervised learning category.

A.10 Prototype Statistics and Their Limitations

A local prototype estimates the mean feature representation of a category on a particular client. Under a fixed feature extractor and independent observations from the same class-conditional distribution,

$$E[P_i^c] = \mu_{i,c,}\ Cov(P_i^c) = \frac{\Sigma_{i,c}}{n_{i,c}},$$

Where $\mu_{i,c}$ is the conditional feature mean, $\Sigma_{i,c}$ is its covariance, and $n_{i,c}$ is the number of local examples. Smaller class samples produce less stable prototype estimates. A prototype computed from 5 examples has five times the variance of one computed from 25 examples, assuming the same within-class covariance.

These assumptions require care. During training, the feature extractor changes, so its feature distribution also changes. Different clients may have different conditional feature distributions, so a local prototype estimates a client-specific mean rather than a universal global mean.

A mean also does not describe the full variation within a category. For feature coordinate r, the empirical variance is

$$s^2_{i,c,r} = \frac{1}{n_{i,c}-1} \sum_{j=1}^{n_{i,c}} (z_{i,j,r} - P^c_{i,r})^2 \ , \ n_{i,c} > 1.$$

For example, the sets {-2,2} and {0,0} both have mean zero, but their population variances are 4 and 0, respectively. Identical class centers can therefore summarize very different distributions. FedTGP's compact communication does not provide the server with the complete within-class feature distribution [1]. Communicating a full covariance matrix for each class would require $K(K+1)/2$ numbers per class, which for $K = 512$ is 131,328 numbers–far more than the 512 numbers required for a prototype.

A.11 Normality Assumptions

Neither CNN training nor prototype averaging requires normally distributed raw pixels. Pixel values are bounded, neighbouring pixels are correlated, and different objects or backgrounds can produce complex distributions. The central limit theorem can sometimes support approximate normality of a sample mean, but only under appropriate conditions. Small local class counts, dependent examples, and changing feature extractors require caution.

A.12 Defining Successful Performance

FedTGP's geometric objective is to learn more useful and separable global prototypes. Its predictive objective is to improve classification across heterogeneous clients [1]. These objectives are related but should be evaluated separately. Larger prototype margins provide useful geometric evidence only when the resulting predictions are also effective.

Client-level-accuracy is

$$A_i = \frac{1}{n_i^{test}} \sum_{j=1}^{n_i^{test}} \Pi(\widehat{y}_{i,j} = y_{i,j}),$$

Here, $\widehat{y}_{i,j}$ is the predicted label, $y_{i,j}$ is the true label, and Π equals 1 for a correct prediction and 0 for otherwise.

Two federation-level summaries are

$$A_{pooled} = \frac{\Sigma_i n_i^{test} A_i}{\Sigma_i n_i^{test}}$$

and

$$A_{client} = \frac{1}{M}\sum_i A_i$$

Pooled accuracy gives equal weight to test examples. Average-client accuracy gives equal weight to participants. They can differ when client test-set sizes are unequal. If one client has 10,000 test examples and another has 100, the pooled accuracy is dominated by the first client, while the average-client accuracy gives equal weight to both.

Accuracy is valid, but frequent categories contribute more to it when the test distribution is imbalanced. Complementary measures can help examine minority-class performance.

For category c,

$$Precision_c = \frac{TP_c}{TP_c + FP_c},$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c}.$$

Balanced accuracy is

$$Balanced\ Accuracy = \frac{1}{C}\sum_{c=1}^{C} Recall_c,$$

And macro-F1 is

$$MacroF1 = \frac{1}{C}\sum_{c=1}^{C} \frac{2Precision_c Recall_c}{Precision_c + Recall_c}$$

These definitions require explicit conventions when classes are absent or denominators are zero. In a federation, the report should state whether metrics are calculated over global classes, locally represented classes, or pooled predictions.

FedTGP reports accuracy comparisons. Additional class-sensitivity metrics would help determine whether gains are distributed across categories [1]. Their absence limits the evidence available about minority-class performance; it does not prove that such performance is poor.

A.13 Scope, Privacy, and Boundaries of the Problem

FedTGP studies labeled image classification over known categories. It supports heterogeneous model architectures and unequal client class distributions through prototype communication [1]. The method keeps original images local and does not require complete client models to be transmitted.

However, reduced data sharing is not equivalent to a formal privacy guarantee. A compressed representation may still reveal information about the data from which it was produced. Prototype communication should therefore be described as limiting direct exposure of raw data and model parameters, rather than guaranteeing protection against every possible attack. Inference attacks, gradient leakage attacks, and membership inference attacks are all potential threats in federated learning, and the paper does not evaluate the method's robustness to these threats.

The experiments also do not establish performance on unknown categories, multilabel tasks, or all forms of domain change. These are boundaries of the evaluated problem. Within those boundaries, the authors report improvements over the compared methods. Whether the experimental protocol, accuracy gains, and ablations adequately support their conclusions should be examined in Phase 3.

B. Methodological Justification

B.1 Introduction to the Methodology

FedTGP was proposed because ordinary federated-learning methods struggle when clients have both **data heterogeneity** and **model heterogeneity** [1]. The authors do not propose a new image-classification neural network. Instead, they propose a new **federated-learning method** that allows different neural-network models to collaborate.

The main idea is that clients do not send raw images or complete model weights to the server. Each client sends compact class representations called local prototypes. The server learns improved trainable global prototypes using Adaptive-Margin-Enhanced Contrastive Learning, or ACL [1].

The following comparison explains how earlier approaches motivate fedTGP's design [1].

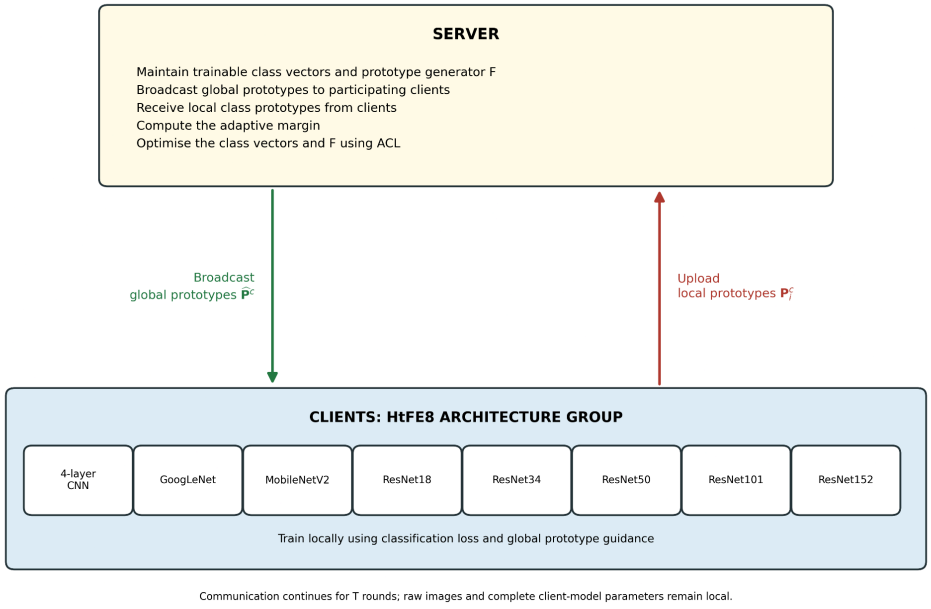


Figure B1: Overview of FedTGP training. Clients train on their local data and upload class prototypes. The server learns global prototypes using ACL and sends them back to guide the next training round. Diagram prepared for this report based on [1].

B.2 Literature Review and Justification

Federated Averaging: FedAvg

FedAvg is the original and most common federated-learning method. Each client trains a local neural network using its own data [2]. It then sends the updated model parameters to a server. The server combines the parameters using weighted averaging.

The aggregation rule is:

$$\theta^{t+1} = \sum_{i \in S_t} \frac{n_i}{\Sigma_{j \in S_t} n_j} \theta_i^{t+1} \qquad \text{(B.1)}$$

In equation (B.1), S_t is the group of clients participating in round t, n_i is the number of samples on client i, and θ_i^{t+1} is its updated model.

FedAvg is appropriate when every client has the same neural-network architecture. For example, if all clients use ResNet18, the server can average corresponding layers and weights [2]. However, FedTGP considers clients using different architectures, including a small CNN, MobileNetV2, GoogLeNet and several ResNet models [1]. Their parameters have different shapes and numbers [1]. Therefore, the authors cannot directly average the model weights.

This is the first reason why the authors did not select FedAvg as their main solution. Instead, they transfer **class-level knowledge** through prototypes [1].

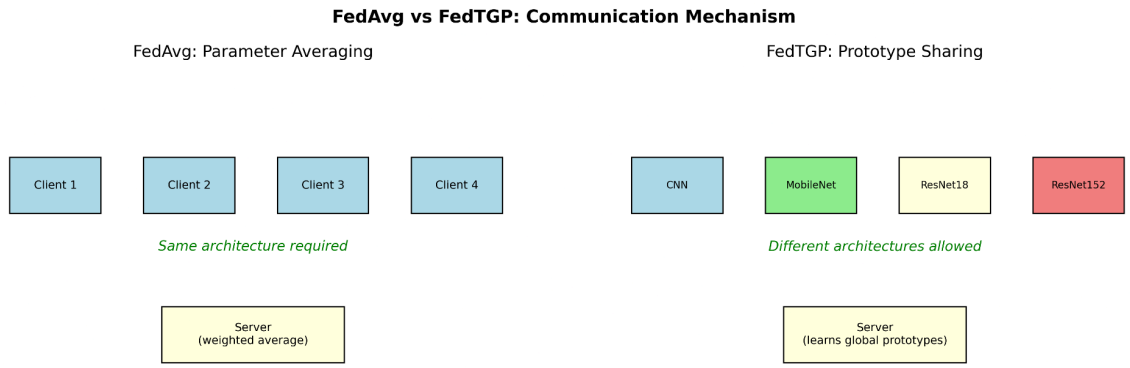


Figure B.2: Communication mechanisms in FedAvg and FedTGP. FedAvg exchanges compatible model parameters, while FedTGP exchanges class prototypes. Author-created illustration based on [1], [2].

Personalized Federated Learning

Personalized federated learning attempts to improve results when clients have different local data distributions. Instead of forcing every client to use exactly the same global model, personalized methods allow each client to maintain a model adapted to its own data.

These methods are helpful for data heterogeneity. For example a client with mostly animal images may need a slightly different model from a client with mostly vehicle images.However, FedTGP authors explain that personalized federated-learning methods alone do not fully solve model heterogeneity [1]. Many still assume compatible model structures or a shared global model framework.

Therefore, the FedTGP authors needed an approach that could support both:

- different local data distributions; and
- genuinely different client architectures

FedTGP uses prototypes because the internal structures of client models can differ, while their [1]outputs can still be mapped into a common feature space.

HeteroFL and FjORD

HeteroFL and FjORD are heterogeneous federated-learning approaches designed for clients with different computational resources [7],[8].

HeteroFL allows weaker clients to train smaller parts of a global model, while stronger clients train larger versions. FjORD uses ordered dropout so that clients can train different-sized submodels.These methods are useful when clients have different memory or computing power. However, the clients still use submodels derived from a common overall architecture. In other words, the models are different in size but are still structurally related.

FedTGP is more flexible because it allows clients to use different architecture families. For example, one client can use MobileNetV2 while another uses ResNet50 [1]. The authors choose prototype communication because it does not require all clients to share a common model design [1].

LG-FedAvg and FedGen

LG-FedAvg separates a client model into local and global parts. The lower layers can remain local, while upper layers are shared or aggregated. FedGen uses a generator to transfer knowledge between heterogeneous client models [1],[9].These approaches are important because they also attempt to support model heterogeneity. However, the FedTGP authors identify limitations [1].

LG-FedAvg still requires some compatible layers to be shared. This creates an architectural dependency. FedGen uses a generator, but its performance depends on whether the generator can produce useful information.In addition, statistical heterogeneity can affect the quality of information that is aggregated or generated [1],[9]. If clients have highly different data distributions, shared layers or generated knowledge may not represent every client equally well.

FedTGP avoids sharing model layers. It shares prototypes instead. A prototype is only a feature [1] vector representing one class, so it is easier to exchange between different architectures.

Knowledge-Distillation Approaches

Knowledge distillation transfers knowledge from one model to another using predictions, probabilities or logits rather than model weights. In federated learning, some knowledge-distillation methods use a public or global dataset.

Clients make predictions for the same public examples, and these predictions are used to transfer knowledge across models.This can support heterogeneity because different architectures can produce class probabilities for the same input. However, the FedTGP authors identify an important limitation: a suitable public dataset may not be available in real applications, especially where the client data is private or specialized [1].

FedDistill

FedDistill shares class-wise logits from clients. A logit is the numerical score produced by a classifier before it is converted into probabilities [10]. The server aggregates the class information and returns it to clients to guide training. FedDistill reduces the need to exchange complete model parameters [10]. However, it shares knowledge at the output-prediction level. The FedTGP authors instead choose feature-level prototypes because they can describe class representations before the final classifier output [1].

FML and FedKD

Federated Mutual Learning or FML, and FedKD use auxiliary models and knowledge distillation to support heterogeneous clients. The exchange of auxiliary-model information rather than relying solely on class prototypes. The fedTGP authors also discuss possible difficulties when both models provide weak learning signals during early training [1].

FedProto and Prototype-Based Learning

FedProto allows clients with different models to collaborate by exchanging class prototypes rather than complete model parameters [11].

A local prototype for class c on client i is calculated as:

$$P_i^c = \frac{1}{|D_{i,c}|} \sum_{(x,y)\in D_{i,c}} f_i\left(x;\theta_i\right), \qquad |D_{i,c}| > 0$$

Here, $D_{i,c}$ is the set of local images belonging to class c, and $f_i\left(x;\theta_i\right)$ is the feature vector extracted from image x.

For example, a client calculates the average feature vector of all its local cat images. This becomes its local cat prototype.

FedProto aggregates local class prototypes to create global prototypes. The server combines local prototypes for each class to obtain a global prototype, which is returned to clients to guide local training [1], [11].

$$\overline{P}^c = \frac{1}{|N_c|} \sum_{i\in N_c} \frac{|D_{i,c}|}{N_c} P_i^c$$

However, the FedTGP authors identify a weakness in simple prototype averaging. They explain that averaging can make the prototypes of different classes less separated. The authors describe this reduction in inter-class separation as prototype-margin shrinkage [1].

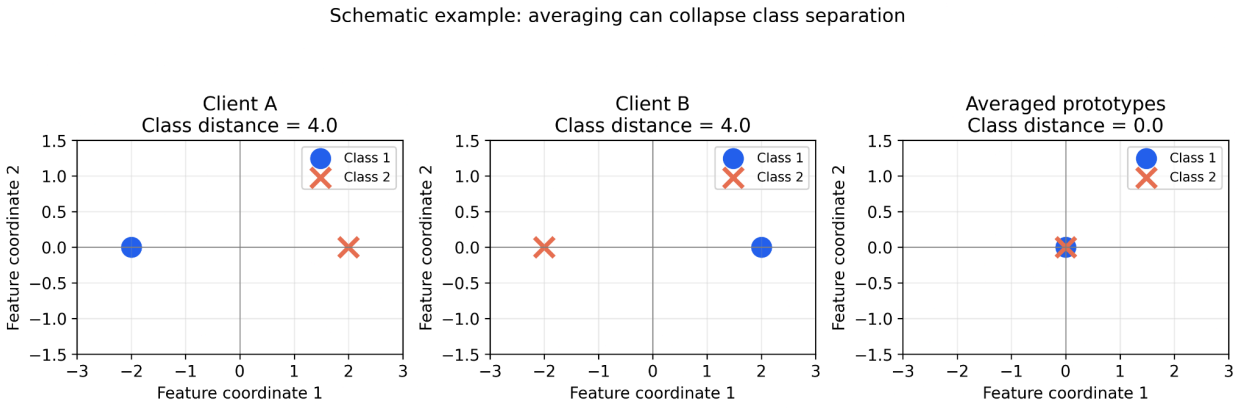


Figure B.3: Schematic example of prototype-margin shrinkage. Two clients separate the classes in opposite directions, so averaging their prototypes makes the global class representation coincide. This is an illustrative example, not a measured FedTGP result.

The example shows why equal feature dimensions alone do not ensure that averaging preserves meaningful class separation.

The authors also argue that a client with more data does not always have a better prototype. Therefore, giving greater weight only because a client has more samples may not always create the most meaningful global prototype. The weighted-averaging protocol also requires clients to upload class-count information $|D_{i,c}|$ for weighting, thereby revealing information about clients' local class distributions [1]. This limitation is the direct research gap addressed by FedTGP [1] .

FedPCL

FedPCL also uses prototypes and contrastive learning. It uses pretrained models and applies contrastive learning on clients to improve their representation space [12]. FedPCL is relevant because it shows that contrastive learning can improve prototype-based federated learning [12]. However, the FedTGP authors state that FedPCL depends on pretrained models. In a real federated-learning environment, clients may have private architectures or limited data, making suitable pretrained models difficult to obtain [1].

FedTGP does not require clients to start with pretrained models. Instead, it learns global prototypes at the server from local prototype information [1].

B.3 Why the Authors Selected Trainable Global Prototypes

Trainable prototypes had previously been used in centralized machine learning to make representations from the same class more compact and to improve the separation between different classes. For domain adaptation, transferring knowledge between domains, similar approaches were also applied. However, these approaches normally require access to centralized data, model parameters or individual feature representations. Such access is unsuitable for federated learning, where clients must keep their data and models private.

FedTGP is an adaptation of trainable prototype learning for the federated setting. The basic prototype averaging, as used by FedProto, does not directly optimize the quality or separation of

global prototypes [1]. As a result, FedTGP generates a trainable vector for each class and processes this vector using a server-side prototype generator [1]:

$$\widehat{P}^{c} = F\left(v^{c}; \theta_{F}\right)$$

Where:

- $\widehat{P}^{c}$ is the generated global prototype for class c;
- v^{c} is a trainable vector associated with class c;
- F is the server-side prototype generator;
- θ_{F} represents the server model parameters.

The generator F is a two-layer fully connected network with a ReLU activation between the layers:

$$F(v^{c}) = W_{2}ReLU(W_{1}v^{c} + b_{1}) + b_{2}$$

Where $W_{1} \in R^{HF \times K}, W_{2} \in R^{K \times HF}$, and H_{F} is the hidden dimension.

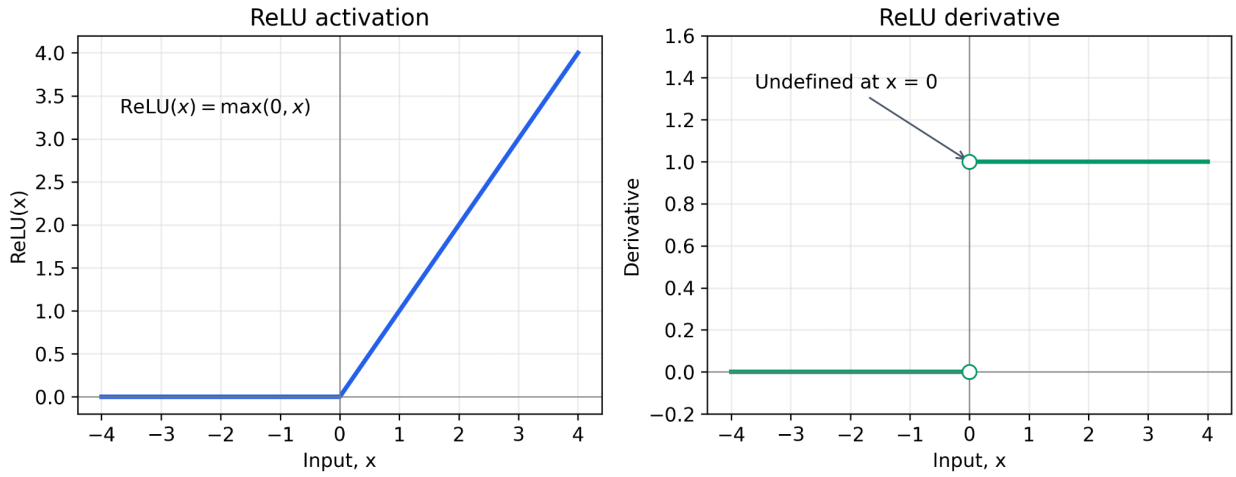


Figure B.4: ReLU and its derivative. Negative inputs produce zero output, while positive inputs pass through unchanged. The mathematical derivative is undefined at 0.

ReLU introduces nonlinearity into the prototype generator, although units receiving consistently negative inputs receive no gradient through this activation.

This design allows the server to optimize global prototypes from uploaded local prototypes without accessing original images or complete client models [1]. Client training and server prototype learning alternate across communication rounds.

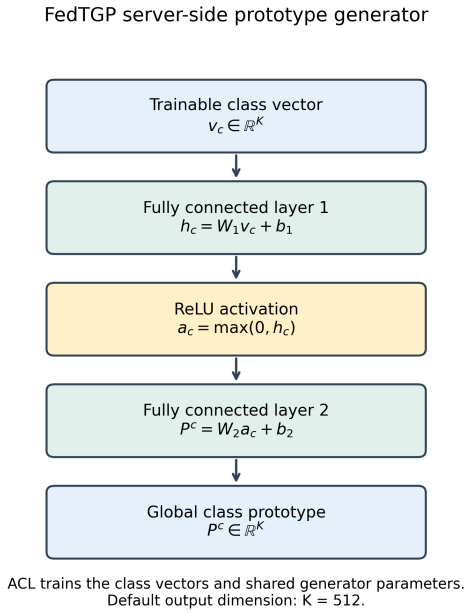


Figure B.5: Server-side prototype generator. A trainable class vector passes through two fully connected layers with a ReLU activation between them to produce a global class prototype. Illustration prepared for this report based on [1].

The server trains both the class vectors and the shared generator using uploaded local prototypes, without accessing the clients' raw images.

B.4 Why the Authors Selected ACL

The authors use Adaptive-Margin-Enhanced Contrastive Learning (ACL) to train the global prototypes [1].The purpose of ACL is to make prototypes of the same class closer together and prototypes of different classes more separate.

The distance between two vectors is calculated using Euclidean distance:

$$\phi(a, b) = ||a - b||_2$$

The adaptive margin used in the paper is:

$$\delta_t = min\,(max_{c \neq ć}\, \phi(Q_t^c, Q_t^{ć}), \tau)$$

Where:

- δ_t is the adaptive margin.
- τ is the maximum allowed margin;
- Q_t^c is the class level center of the participating client's prototypes for class c.

The margin is adaptive because it changes during training according to the current arrangement of class prototypes. It is limited by τ to avoid becoming excessively large.

The ACL objective for one local prototype is:

$$l_{i,c} = -log \frac{e^{-[d(P_i^c, \widehat{P}^c) + \delta_t]}}{e^{-[d(P_i^c, \widehat{P}^c) + \delta_t]} + \Sigma_{ć \neq c} e^{-d(P_i^c, \widehat{P}^{ć})}}$$

In simple terms, this objective teaches the server that:

- A local cat prototype should be close to the global cat prototype;
- It should be far from global dog, horse and truck prototypes;
- The separation requirement should adapt as training progresses.

The authors selected ACL because it directly addresses the prototype-margin-shrinkage problem [1]. ACL is intended to create global prototypes that are more discriminative than those produced by simple averaging [1].

In the settings evaluated , the paper's results show up to 18.96 percentage points better than FedProto [1]. This result is for the given configuration and cannot ensure that it performs best in all other cases.

B.5 Local Training Objective and Prediction

Each client trains with a combination of classification loss and prototype-alignment loss:

$$L_i = L_{classification} + \lambda L_{alignment}$$

The classification loss encourages the local model to predict correct image labels. The alignment loss encourages local class representations to become close to the corresponding global prototypes.

The paper uses:

$$\lambda = 0.1$$

as the default alignment weight .

The prototype term has a coefficient of $\lambda = 0.1$, but its actual influence depends on the scales, reductions and gradients of both loss terms [1]. The coefficient alone does not show which term dominates training.

Prototype based accuracy evaluates feature extractor together with the global prototype decision rule. It does not independently establish the predictive quality of the classification head. Comparing both prediction approaches would help clarify the source of the improvement.

B.6 Client Models Used in FedTGP

The paper's main heterogeneous model group is named HtFE8. It contains a four-layer CNN, GoogLeNet, MobileNetV2, ResNet18, ResNet34, ResNet50, ResNet101 and ResNet152 [1].

Four-layer CNN

The four-layer CNN is a relatively simple convolutional neural network. CNNs are designed for images because convolution layers learn local patterns such as edges, colors, textures and shapes [1]. This model represents a client with limited computational capability.

GoogLeNet

GoogLeNet uses Inception modules.These modules process the same input through different branches, including different convolution sizes and pooling operations. It is included because it extracts image features differently from basic CNNs and ResNet models.

MobileNetV2

MobileNetV2 is a lightweight and efficient neural network architecture. It uses inverted residual structures and linear bottleneck layers [1]. It represents devices with limited processing power, such as mobile or edge devices.

ResNet18

ResNet18 is a relatively shallow residual network. Residual connections help networks train effectively by allowing layers to learn changes relative to their inputs [1]. It represents a moderate-complexity client model.

The defining feature of ResNet is the residual block:

$$y = F(X, \{W_i\}) + X$$

The gradient flow through a residual block is:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y}\left(1 + \frac{\partial F(X)}{\partial X}\right)$$

The identity term 1 ensures that gradients can flow directly to earlier layers without passing through the potentially vanishing derivatives of F. This is why ResNets can be trained with many layers [1].

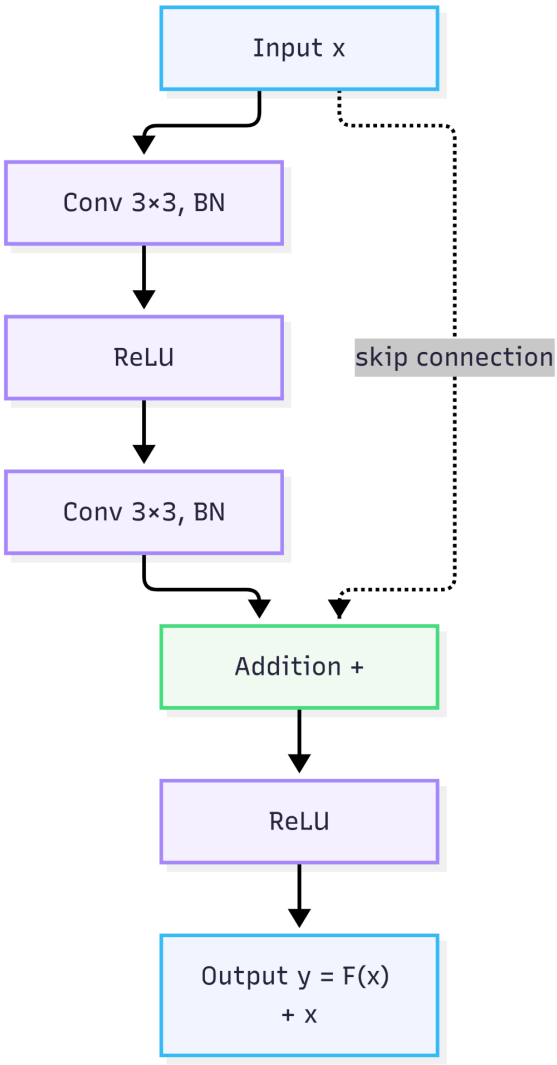


Figure B.6: Residual block. The skip connection provides a direct path for information and gradient propagation.

ResNet34

ResNet34 is deeper than ResNet18 and provides greater model capacity, although this does not guarantee better performance [1].

ResNet50

ResNet50 is a deeper residual architecture using bottleneck blocks. It represents a more powerful client model [1].

ResNet101

ResNet101 is substantially deeper than ResNet50. It represents a high-capacity feature extractor [1].

ResNet152

ResNet152 is the deepest ResNet in the main HtFE8 group. It creates a strong contrast with the [1] small CNN and MobileNetV2 [1].

The purpose of these models is not to decide which architecture is best. Their purpose is to create a difficult heterogeneous environment. FedTGP tests whether these different models can collaborate through shared 512-dimensional prototypes.

B.7 FedTGP Methodology Diagram

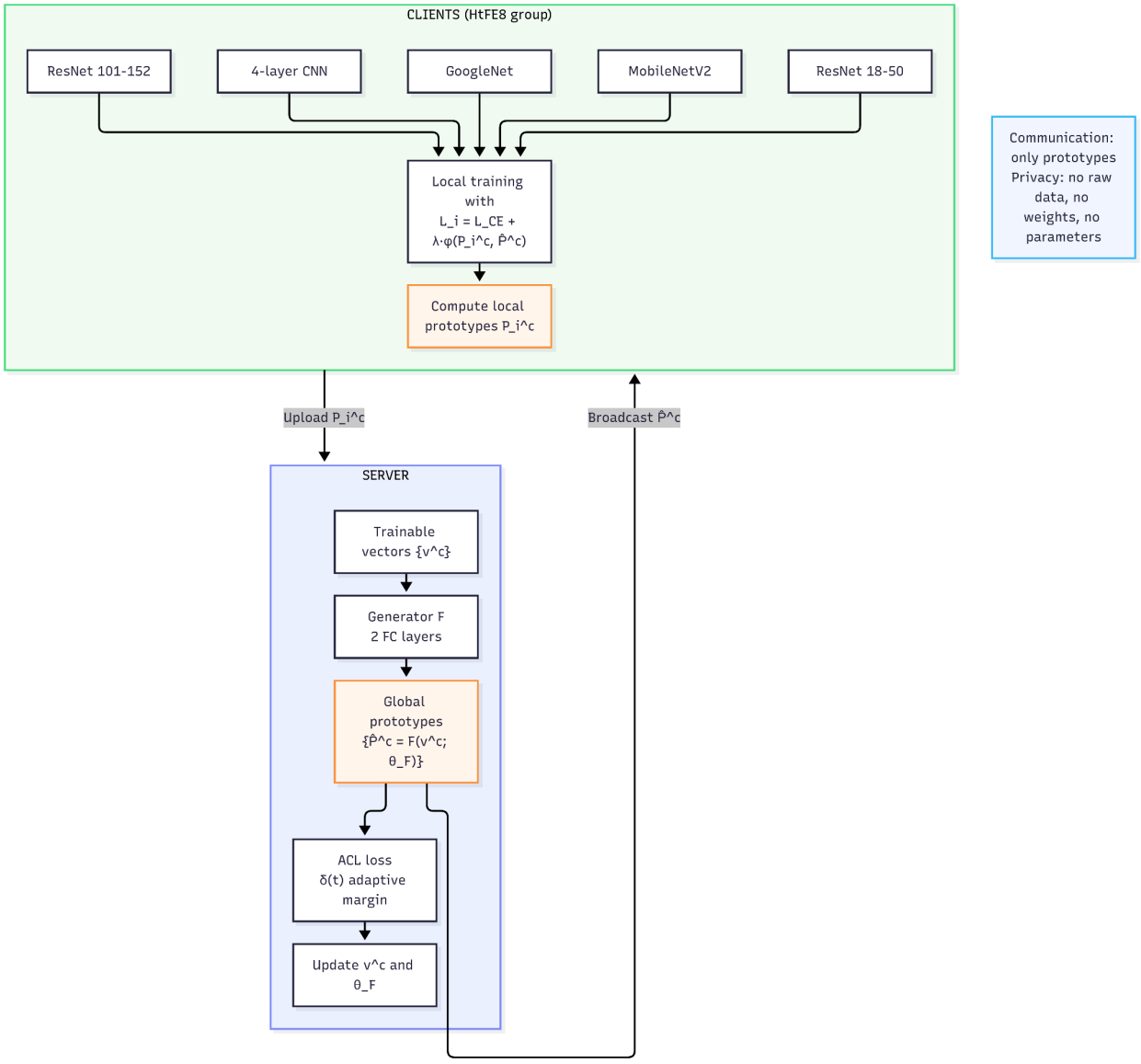


Figure B.7: *FedTGP training cycle*. Clients upload local prototypes, and the server learns global prototypes that guide subsequent local training [1].

C. Data Pipeline Analysis

C.1 Introduction

FedTGP method is evaluated on four supervised image classification datasets: CIFAR-10, CIFAR-100, Flowers102 and Tiny-ImageNet [1]. These datasets differ in image size, number of classes, class balance and classification difficulty.

The data pipeline is crucial because FedTGP is not a single model trained with centrally collected data [1]. Rather, the images are spread across 20 federated clients. The authors set up non-IID distributions where different classes and different class proportions are assigned to different clients. Thus, image preprocessing, client-level partitioning, local train/test splitting and treatment of class imbalance are also components of the pipeline.

This section distinguishes the pipeline described in the paper from the configuration observed in the linked repository. It examines the consequences of preprocessing, partitioning and splitting choices rather than treating every implementation choice as an error [1], [5].

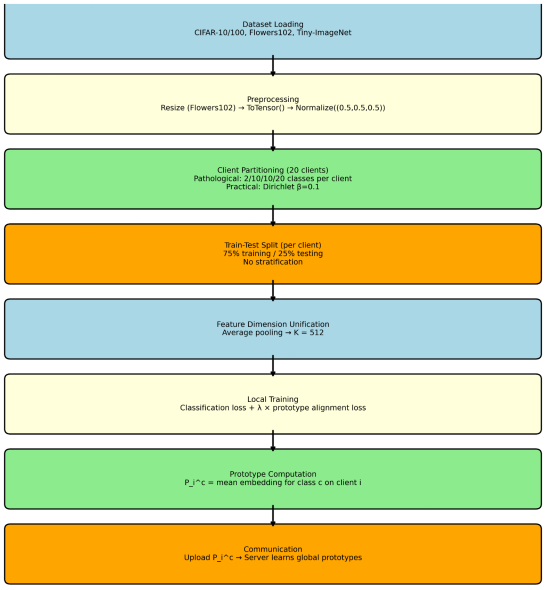


Figure C.1: Pipeline reconstructed from the paper and inspected repository. The stages include preprocessing, client allocation, local splitting and training [1], [5].

C.2 Overview of the Datasets

The original dataset characteristics are discussed in Section A.7. This section examines how those images are transformed, allocated to clients and divided into local training and test sets.

C.3 Preprocessing & Feature Scaling

The FedTGP paper reports the experimental datasets and federated settings but does not list every transformation [1]. Therefore, the following preprocessing details were obtained from the PFLlib dataset-generation code linked by the official FedTGP repository [5], [13]. The repository also refers users to HtFLlib for the required environment , and heterogeneous federated-learning resources [13], [14].

For CIFAR-10 and CIFAR-100, the linked generators use $To\ Tensor()$ to convert each image into a PyTorch tensor [5]. This also converts the pixel values from the original range of 0-255 to

values approximately between 0 and 1 [5]. The generators then apply $Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))$ to the three RGB channels.

For each colour channel, this transformation can be expressed as:

$$x_{normalized,k} = \frac{x_k - 0.5}{0.5} = 2x_k - 1$$

Where x_k is the converted tensor value of the pixel. As a result, the pixel values are mapped approximately from $([0, 1])$to $([-1, 1])$.

The inspected Flowers102 generator resizes images to 64×64, converts them to tensors and applies normalization [5]. Resizing supports consistent batching but may remove useful visual details. Fixed normalization constants of 0.5 map the converted pixel range to [-1,1]; they do not guarantee zero empirical mean or unit variance. These are preprocessing trade-offs. The inspected configuration should be distinguished from the exact configuration used in the published experiments.

The Tiny-ImageNet generator also incorporates tensor conversion and normalization. The inspected generation scripts do not include additional random transformations, such as random cropping, rotation or horizontal flipping [5] . Thus, in this step the pipeline does not explicitly use training data augmentation, but rather transforms and scales it.

Normalization is useful because the neural network is sensitive to the magnitude of the features it is presented with [5].

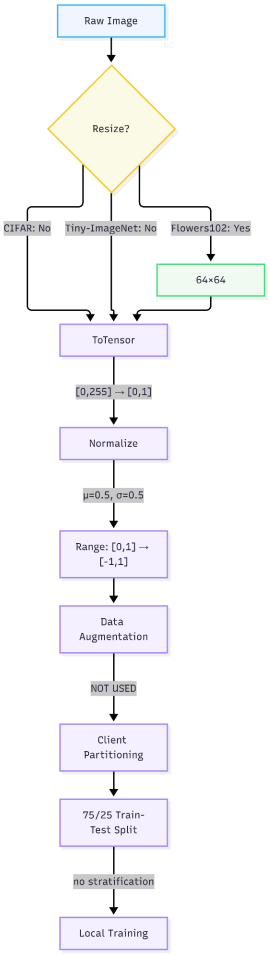


Figure C.2: Preprocessing stages reconstructed from the inspected PFLlib scripts. Highlighted stages identify review considerations, rather than proven errors [5].

C.4 Label Encoding

All four datasets use integer-encoded class labels.

CIFAR-10 uses IDs from 0 to 9. CIFAR-100 uses 0 to 99. Flowers102 uses 0 to 101. Tiny-Imagenet uses 0 to 199. The linked pipeline loads these integer labels directly, without putting them into one-hot encoding [5].

Integer labels are suitable for a multiclass cross-entropy loss in PyTorch. All clients must have the same class map [5]. Otherwise, a class label could have different meanings on different clients, resulting in incorrect local and global prototypes.

There is no specific imputation of missing values. This is reasonable because the inputs are established image datasets rather than tabular records containing empty numerical cells [5]. The pipeline uses the official dataset loaders to download or read the images and labels. But it does not have an individual procedure for missing images and corrupted files [5].

The absence of imputation is not itself a weakness for these image datasets. However, using loaders does not demonstrate that file integrity and image-label consistency were explicitly checked. Incorrect labels would affect supervised training and prototype construction. This is a limitation in the documented validation procedure,not evidence that such errors occurred.

C.5 Class Imbalance and Client Data Distribution

The paper primarily studies statistical heterogeneity through non-IID level distributions, which produce severe client-level class imbalance. It does not use SMOTE, oversampling or synthetic data generation.The paper uses two distribution settings.

Pathological non-IID setting

In the pathological setting, each client receives examples from only a limited number of classes.

For CIFAR-10, each client receives only two classes from the ten available classes. For CIFAR-100, each client receives ten classes. For Flowers102, each client receives ten classes. For Tiny-ImageNet, each client receives twenty classes.This creates severe data heterogeneity because a client may never see many classes during local training.

Practical non-IID setting

In the practical setting, the authors use a Dirichlet distribution:

$$q_c = (q_{c,1,\ldots,} q_c, M) \sim Dirichlet(\beta 1_M),$$

With

$$\beta = 0.1$$

A low value of β produces highly uneven class proportions. For example, one client may receive many examples of class "truck" while another client receives only a few or none.

CIFAR-10 and CIFAR-100 are balanced globally, but their local client datasets become imbalanced after this distribution. Flowers102 has both original class imbalance and additional client-level imbalance.

The authors adopt prototype sharing for learning under this circumstance [1]. But the partition parameter is not enough to describe the imbalance realised. Unstable estimates of prototype categories can be obtained in small class samples, and categories with fewer clients will have less diverse cross-client information. Client-level class counts would provide for easier assessment of these conditions. Prototype sharing does not ensure the uniformity of performance of clients or categories.

C.6 Train-Test Splitting

After the data have been partitioned among clients, each client's private data are divided into:

75% training data

and

25% testing data

The linked implementation uses *train_test_split* with *train_size = 0.75* and shuffling enabled [5]. The training portion updates the client's local model, while the test portion evaluates the model using unseen local samples.

The parameter *stratify* is not directly used by the splitting function. Hence, it does not ensure that all the classes appearing in a client will be proportionally represented in both the training and test sets [5]. This is especially true if the client has just a few samples in a class .

If any methods are implemented, the same client partitions and train-test samples should be used. Otherwise performance differences could be due to data allocation instead of learning algorithms.

The paper selects the best average client-test result across communication rounds [1]. Although test examples are not directly used for gradient updates, selecting a round using test performance can produce an optimistic estimate. A separate validation set would support round selection while reserving the test set for final evaluation.

C.7 Dataset Choice for Phase 2

For the Phase 2 experiment, CIFAR-10 is selected due to its ten categories and the small size of its images. FedProto will serve as the baseline method. Comparisons should be done using the same data partitions and evaluation process; architectures, communication rounds and random seeds should be documented. The coursework results will not be presented as an exact reproduction of the experimental set up in the paper as differences from the paper's configuration will be explicitly mentioned.

References

[1] J. Zhang, Y. Liu, Y. Hua, and J. Cao, "FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning," *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 15, pp. 16768–16776, Mar. 2024, doi: 10.1609/aaai.v38i15.29617.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artificial Intelligence and Statistics, vol. 54, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[3] "CIFAR-10 and CIFAR-100 datasets." Accessed: Sep. 14, 2026. [Online]. Available: https://cave.cs.toronto.edu/kriz/cifar.html

[4] P. Chrabaszcz, I. Loshchilov, and F. Hutter, “A Downsampled Variant of ImageNet as an Alternative to the CIFAR Datasets,” 2017, *arXiv*. doi: 10.48550/ARXIV.1707.08819.

[5] TsingZ0, *TsingZ0/PFLlib*. (Sep. 12, 2026). Python. Accessed: Sep. 14, 2026. [Online]. Available: https://github.com/TsingZ0/PFLlib

[6] M.-E. Nilsback and A. Zisserman, “Automated Flower Classification over a Large Number of Classes,” in *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, Bhubaneswar, India: IEEE, Dec. 2008, pp. 722–729. doi: 10.1109/ICVGIP.2008.47.

[7] E. Diao, J. Ding, and V. Tarokh, “HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients,” 2020, *arXiv*. doi: 10.48550/ARXIV.2010.01264.

[8] S. Horvath, S. Laskaridis, M. Almeida, I. Leontiadis, S. I. Venieris, and N. D. Lane, “FjORD: Fair and Accurate Federated Learning under heterogeneous targets with Ordered Dropout,” 2021, *arXiv*. doi: 10.48550/ARXIV.2102.13451.

[9] Z. Zhu, J. Hong, and J. Zhou, “Data-Free Knowledge Distillation for Heterogeneous Federated Learning,” 2021, doi: 10.48550/ARXIV.2105.10056.

[10] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, “Communication-Efficient On-Device Machine Learning: Federated Distillation and Augmentation under Non-IID Private Data,” 2018, *arXiv*. doi: 10.48550/ARXIV.1811.11479.

[11] Y. Tan *et al.*, “FedProto: Federated Prototype Learning across Heterogeneous Clients,” *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, pp. 8432–8440, Jun. 2022, doi: 10.1609/aaai.v36i8.20819.

[12] Y. Tan, G. Long, J. Ma, L. Liu, T. Zhou, and J. Jiang, “Federated Learning from Pre-Trained Models: A Contrastive Learning Approach,” 2022, *arXiv*. doi: 10.48550/ARXIV.2209.10083.

[13] TsingZ0, *TsingZ0/FedTGP*. (Jul. 27, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/FedTGP

[14] TsingZ0, *TsingZ0/HtFLlib*. (Sep. 08, 2026). Python. Accessed: Sep. 15, 2026. [Online]. Available: https://github.com/TsingZ0/HtFLlib

[15] Visual Geometry Group, University of Oxford, “102 category flower dataset: Category statistics.” Accessed: Oct. 12, 2026. [Online]. Available: https://www.robots.ox.ac.uk/~vgg/data/flowers/102/categories.html

[16] TensorFlow Datasets, “Oxford Flowers 102.” Accessed: Oct.12, 2026. [Online]. Available: https://www.tensorflow.org/datasets/catalog/oxford_flowers102

[17] J. Wu, Q. Zhang, and G. Xu, Tiny ImageNet challenge, CS231n project report, Stanford University, 2017. [Online]. Available: https://cs231n.stanford.edu/reports/2017/pdfs/930.pdf